Genomics Project Report

Mechanisms of Action (MoA) Prediction

Multi-label Drug Classification with Deep Learning

Kasra Korminejad

April 2026

Abstract

We address the LISH-MoA Kaggle competition: predicting which of 206 biological mechanisms of action (MoA) a drug activates, given gene expression and cell viability measurements. This is a multi-label binary classification problem evaluated by mean column-wise log loss. We implemented and compared two deep learning experiments. **Experiment 1** established a baseline multi-layer perceptron (MLP) with minimal preprocessing, reaching a best validation binary cross-entropy (BCE) loss of 0.0207 but showing significant overfitting. **Experiment 2** addresses these shortcomings through quantile normalization, PCA-based dimensionality reduction, a deeper residual architecture with batch normalization, label smoothing, multi-task learning with non-scored targets, and cosine annealing with early stopping. Experiment 2 is the recommended model going forward.

Contents

1 Introduction **3**

2 Dataset **3**
2.1 Files and Sizes . . . 3
2.2 Feature Columns . . . 3
2.3 Target Columns . . . 4
2.4 Class Imbalance . . . 4
2.5 Control Samples . . . 4

3 Data Processing **4**
3.1 Experiment 1 — Minimal Preprocessing . . . 4
3.2 Experiment 2 — Full Preprocessing Pipeline . . . 5
3.2.1 Metadata Encoding . . . 5
3.2.2 Quantile Normalisation . . . 5
3.2.3 PCA Dimensionality Reduction . . . 5
3.2.4 Drug-Grouped Cross-Validation . . . 5

4 Evaluation Metrics **6**
4.1 Competition Metric: Mean Column-wise Log Loss . . . 6
4.2 BCE Loss vs. MoA Log Loss . . . 6

4.3 Reference Baselines . . . 7

5 Model Architecture **7**
5.1 Experiment 1 — Baseline MLP . . . 7
5.2 Experiment 2 — BN-MLP with Residual Block and Dual Heads . . . 8
5.2.1 Overall Data Flow . . . 8
5.2.2 Batch Normalisation . . . 8
5.2.3 SiLU Activation . . . 8
5.2.4 Residual Block . . . 9
5.2.5 Dual Output Heads and Multi-task Learning . . . 9
5.2.6 Kaiming Initialisation . . . 9
5.2.7 Dropout 0.3 . . . 9

6 Training Configuration **9**
6.1 Experiment 1 . . . 9
6.2 Experiment 2 . . . 10
6.2.1 AdamW vs. Adam . . . 10
6.2.2 Label Smoothing . . . 11
6.2.3 Cosine Annealing . . . 11
6.2.4 Early Stopping . . . 11
6.2.5 Total Loss . . . 11

7 Results **11**
7.1 Experiment 1 . . . 11
7.2 Experiment 2 . . . 12

8 Future Work and Takeaways **12**
8.1 Immediate Next Steps . . . 12
8.2 Modelling Improvements . . . 13
8.3 Key Takeaways . . . 14

9 Conclusion **14**

1 Introduction

The Connectivity Map project (Broad Institute, LISH, NIH LINCS) aims to accelerate drug discovery by automating the prediction of a compound's mechanism of action from its cellular signature. In this competition, each drug sample is characterised by 772 gene expression measurements and 100 cell viability measurements taken across a pool of 100 human cell types. The goal is to predict, for each sample, which of 206 binary MoA labels are active.

Key properties that make this problem challenging:

- **Extreme class imbalance.** Across 206 targets, the mean positive rate is only 0.34%. 173 out of 206 targets have a positive rate below 0.5%, and only 2 targets exceed 3%.
- **High dimensionality.** The raw feature space has 876 columns (4 metadata + 772 gene + 100 cell), with substantial redundancy and noise.
- **Drug-level leakage risk.** Multiple samples (different dose and time combinations) share the same underlying drug. Naive cross-validation that splits by sample rather than drug leaks information and inflates validation scores.
- **Multi-label output.** Each sample may activate zero, one, or many MoA targets simultaneously.

2 Dataset

2.1 Files and Sizes

Table 1: Dataset files and dimensions.

File	Role	Rows	Columns
`train_features.csv`	Input features	23,814	876
`test_features.csv`	Input features	3,982	876
`train_targets_scored.csv`	Competition targets	23,814	207 (206 + id)
`train_targets_nonscored.csv`	Auxiliary targets	23,814	403 (402 + id)
`train_drug.csv`	Drug grouping	23,814	2
`sample_submission.csv`	Submission template	3,982	207

2.2 Feature Columns

Each row represents one experimental sample identified by `sig_id`. The 876 feature columns break down as:

- `cp_type` — treatment type: `trt_cp` (drug treatment) or `ctl_vehicle` (negative control).
- `cp_time` — treatment duration: 24, 48, or 72 hours.
- `cp_dose` — dose level: `D1` (low) or `D2` (high).
- `g-0` to `g-771` — 772 normalised gene expression measurements.
- `c-0` to `c-99` — 100 normalised cell viability measurements.

2.3 Target Columns

`train_targets_scored.csv` contains 206 binary MoA labels that are evaluated and submitted. `train_targets_nonscored.csv` contains 402 additional binary labels which are available during training but excluded from the competition score.

2.4 Class Imbalance

The target distribution is highly skewed. Table 2 summarises the positive rate statistics across all 206 scored targets.

Table 2: Positive rate statistics across 206 scored MoA targets (treatment samples only).

Statistic	**Value**
Minimum positive rate	0.004%
Median positive rate	0.162%
Mean positive rate	0.343%
Maximum positive rate	3.494%
Targets with positive rate $< 0.5\%$	173 / 206
Targets with positive rate $< 1.0\%$	184 / 206
Targets with positive rate $> 1.0\%$	22 / 206
Targets with positive rate $> 3.0\%$	2 / 206

2.5 Control Samples

Samples with `cp_type = ctl_vehicle` are negative controls. By construction, all 206 MoA labels are zero for these samples — they receive no drug. There are 1,866 control samples in training (7.8%) and 358 in test (9.0%). These are handled explicitly at inference: rather than running them through a model that might assign nonzero probability, their predictions are hard-set to zero.

3 Data Processing

3.1 Experiment 1 — Minimal Preprocessing

In Experiment 1, preprocessing was limited to:

1. **Control filtering.** Control samples (`cp_type = ctl_vehicle`) were removed from the training set entirely, so the model only trained on treatment samples.
2. **Categorical encoding.** `cp_dose` was mapped to a binary variable (`D1` $\rightarrow$ 0, `D2` $\rightarrow$ 1). `cp_time` and `cp_type` were passed as-is (numeric or left as strings, leaking implicit ordinal assumptions).
3. **No normalisation.** The 772 gene and 100 cell features were fed raw into the network.

This resulted in an input dimensionality of 873 (870 numeric feature columns + `cp_dose_bin` + `cp_time` + no metadata).

3.2 Experiment 2 — Full Preprocessing Pipeline

Experiment 2 applied a systematic pipeline designed to clean the distribution and reduce dimensionality.

3.2.1 Metadata Encoding

- `cp_type` → binary (1 if `trt_cp`, else 0).
- `cp_time` → continuous, normalised to $[0, 1]$ by dividing by 72.
- `cp_dose` → binary (`D1` → 0, `D2` → 1).

3.2.2 Quantile Normalisation

Gene expression and cell viability features were independently transformed using `QuantileTransformer` with a Gaussian output distribution. This maps each feature's empirical distribution to $\mathcal{N}(0, 1)$, regardless of the original shape. The transformer was fit exclusively on the training set and applied to the test set, preventing data leakage.

The motivation is twofold: (1) outlier values — common in biological assays — no longer dominate the gradient signal; (2) the Gaussian output matches the implicit assumptions of batch normalisation, which expects approximately centred activations.

3.2.3 PCA Dimensionality Reduction

Principal Component Analysis (PCA) was applied separately to the quantile-normalised gene features (772 dimensions) and cell features (100 dimensions), retaining enough components to explain 95% of the total variance. This reduced the gene features from 772 to approximately 602 components and cell features from 100 to 83 components. The PCA was fit on training data only.

Both the original quantile-normalised features and the PCA-reduced features were concatenated into the final input vector. This gives the model access to the compact principal structure (via PCA) and the fine-grained individual-feature information simultaneously. The final input dimension is 1,560:

$$\underbrace{3}_{\text{metadata}} + \underbrace{602}_{\text{gene PCA}} + \underbrace{83}_{\text{cell PCA}} + \underbrace{772}_{\text{gene QT}} + \underbrace{100}_{\text{cell QT}} = 1{,}560$$

3.2.4 Drug-Grouped Cross-Validation

Both experiments use `MultilabelStratifiedKFold` with $k = 5$ folds. However, the split is performed at the *drug level* rather than the sample level. Each drug appears in multiple samples (different doses and times), so a naive sample-level split would place the same drug in both training and validation, leaking its chemical identity. The correct procedure is:

1. Aggregate targets to the drug level (a drug is positive for an MoA if any of its experiments are positive).
2. Assign each drug to exactly one fold.
3. Map drug-level fold assignments back to individual samples via `train_drug.csv`.

This ensures that every sample in a validation fold comes from a drug the model has never seen in training.

4 Evaluation Metrics

4.1 Competition Metric: Mean Column-wise Log Loss

The official competition metric is the **mean column-wise log loss** across all 206 scored targets:

$$\mathcal{L}_{\text{MoA}} = -\frac{1}{M}\sum_{m=1}^{M}\frac{1}{N}\sum_{i=1}^{N}\left[y_{i,m}\log\hat{y}_{i,m} + (1-y_{i,m})\log(1-\hat{y}_{i,m})\right] \tag{1}$$

where N is the number of test samples, $M = 206$ is the number of scored targets, $y_{i,m} \in \{0,1\}$ is the ground truth, and $\hat{y}_{i,m} \in (0,1)$ is the predicted probability. A **lower score is better.**

Key properties of this metric:

- **Equal weight per column.** A rare target with 0.04% positive rate is weighted identically to the most common target (3.49%). This means the model cannot succeed by mastering only the common MoAs — every column must be calibrated well.
- **Penalises confidence in wrong answers.** Predicting $\hat{y} = 0.01$ for a true positive contributes $-\log(0.01) \approx 4.6$ to the loss, whereas $\hat{y} = 0.5$ contributes only $-\log(0.5) \approx 0.69$. Overconfident wrong predictions are severely punished.
- **Requires calibrated probabilities.** Predictions are clipped to $[\epsilon, 1-\epsilon]$ with $\epsilon = 10^{-15}$ before evaluation. The model must output well-calibrated probabilities, not just binary decisions.

4.2 BCE Loss vs. MoA Log Loss

The **binary cross-entropy (BCE) loss** used during training is:

$$\mathcal{L}_{\text{BCE}} = -\frac{1}{N\cdot M}\sum_{i=1}^{N}\sum_{m=1}^{M}\left[y_{i,m}\log\hat{y}_{i,m} + (1-y_{i,m})\log(1-\hat{y}_{i,m})\right] \tag{2}$$

This is mathematically the same expression, but BCE averages over *all $N \times M$ elements jointly*, treating every cell in the prediction matrix equally. The competition metric averages *first within each column, then across columns*, giving equal weight to each MoA regardless of how many samples are positive. The distinction matters when positive rates vary widely across columns: in BCE, the many zero-labeled cells of rare targets are averaged in with the more-balanced common targets, diluting the signal from the hardest columns. Monitoring the competition metric during validation — not just BCE — is essential to selecting the best model.

Table 3: Reference score points for the competition metric.

Strategy	MoA Log Loss
Predict 0.5 for every sample and target	~ 0.693
Predict the column-wise mean positive rate	~ 0.022
Experiment 1 (baseline MLP)	$\sim 0.021^*$
Experiment 2 (improved MLP)	TBD after full run
Kaggle top leaderboard	~ 0.013

*Estimated from val

BCE loss; the competition metric was not tracked during exp1 training.

4.3 Reference Baselines

The "predict column mean" baseline (~ 0.022) is the critical threshold: any model that cannot beat this is providing no useful information beyond knowing the prior positive rates.

5 Model Architecture

5.1 Experiment 1 — Baseline MLP

The first model is a simple three-layer feedforward network:

$$\mathbf{h}_1 = \mathrm{ReLU}(\mathbf{W}_1\mathbf{x}), \quad \mathbf{h}_1' = \mathrm{Dropout}_{0.5}(\mathbf{h}_1) \tag{3}$$

$$\mathbf{h}_2 = \mathrm{ReLU}(\mathbf{W}_2\mathbf{h}_1'), \quad \mathbf{h}_2' = \mathrm{Dropout}_{0.5}(\mathbf{h}_2) \tag{4}$$

$$\hat{\mathbf{y}} = \mathbf{W}_3\mathbf{h}_2' \tag{5}$$

with layer dimensions $873 \to 512 \to 256 \to 206$. The output is raw logits; the sigmoid is applied externally when computing probabilities.

Limitations identified:

- **No batch normalisation.** Without normalised activations, gradient magnitudes vary across layers, making learning unstable at depth.
- **ReLU dying neurons.** With dropout 0.5, a substantial fraction of neurons can become permanently inactive, reducing effective model capacity.
- **Dropout 0.5 too aggressive.** The combination of no batch normalisation and 50% dropout forces the model to rely on very few active features per forward pass, which — combined with the high-dimensional, sparse target space — causes severe underfitting on validation.
- **No multi-task learning.** The 402 non-scored targets, which share the same biological substrate, were ignored entirely.
- **Default weight initialisation.** PyTorch's default initialisation (uniform) is not calibrated for ReLU activations, causing early training instability.

5.2 Experiment 2 — BN-MLP with Residual Block and Dual Heads

The second model addresses every limitation of Experiment 1.

5.2.1 Overall Data Flow

```
Input (1560-dim)
        |
        v
 [Linear 1560->1024] -> [BatchNorm] -> [SiLU] -> [Dropout 0.3]
        |
 [Linear 1024->512 ] -> [BatchNorm] -> [SiLU] -> [Dropout 0.3]
        |
 [Linear  512->256 ] -> [BatchNorm] -> [SiLU] -> [Dropout 0.3]
        |
 [ResidualBlock 256]:
   x -> Linear -> BN -> SiLU -> Dropout -> Linear -> BN
       + x (skip connection) -> SiLU
        |
   +----+----+
   |         |
head_scored  head_aux
Linear->206  Linear->402
```

5.2.2 Batch Normalisation

Batch normalisation [1] is applied after each linear layer, before the activation. It normalises the pre-activation distribution within each mini-batch:

$$\hat{x}_i = \frac{x_i - \mu_{\mathcal{B}}}{\sqrt{\sigma_{\mathcal{B}}^2 + \epsilon}}, \quad y_i = \gamma \hat{x}_i + \beta \tag{6}$$

where $\mu_{\mathcal{B}}$ and $\sigma_{\mathcal{B}}^2$ are the batch mean and variance, and γ, β are learnable parameters. This keeps gradient magnitudes stable across layers, dramatically reduces sensitivity to learning rate, and allows the model to train effectively at greater depth. It also acts as a regulariser, reducing the need for large dropout rates.

5.2.3 SiLU Activation

The Sigmoid Linear Unit (SiLU, also known as Swish) is defined as:

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}} \tag{7}$$

Unlike ReLU, SiLU is smooth, non-monotonic, and has a small negative output for slightly negative inputs. This prevents the dying neuron problem and produces more uniform gradient flow through the network. Empirically, SiLU outperforms ReLU on tabular biological datasets.

5.2.4 Residual Block

After the backbone, a residual block [2] is added:

$$\mathbf{h}_{\text{out}} = \text{SiLU}(\mathbf{h} + f(\mathbf{h})) \tag{8}$$

where $f(\mathbf{h}) = \text{BN}(\mathbf{W}_2 \cdot \text{Drop}(\text{SiLU}(\text{BN}(\mathbf{W}_1\mathbf{h}))))$. The skip connection $\mathbf{h}$ bypasses the transformation. If the residual path learns nothing ($f(\mathbf{h}) \to \mathbf{0}$), the block degenerates to an identity mapping — adding depth cannot hurt. The skip connection also shortens the effective gradient path from output to input, mitigating vanishing gradients.

5.2.5 Dual Output Heads and Multi-task Learning

The backbone is shared between two linear output heads:

- **head_scored**: $\mathbb{R}^{256} \to \mathbb{R}^{206}$ — predicts the competition targets.
- **head_aux**: $\mathbb{R}^{256} \to \mathbb{R}^{402}$ — predicts the non-scored targets, active only during training.

The 402 non-scored MoA labels describe different biological activities of the same drugs, measured from the same gene/cell signals. Training the shared backbone to predict both sets forces it to learn a richer internal representation than would be possible from 206 targets alone. The auxiliary head is discarded at inference; it acts purely as a structured regulariser.

5.2.6 Kaiming Initialisation

All linear layers are initialised using Kaiming (He) initialisation [3]:

$$W \sim \mathcal{N}\left(0, \sqrt{\frac{2}{\text{fan_in}}}\right) \tag{9}$$

This keeps the variance of activations approximately constant through a ReLU/SiLU network at initialisation, preventing saturation or collapse before training begins.

5.2.7 Dropout 0.3

Dropout was reduced from 0.5 (Experiment 1) to 0.3. With batch normalisation already providing implicit regularisation by adding noise through the mini-batch statistics, aggressive dropout is counterproductive. Dropout 0.3 prevents co-adaptation without unnecessarily crippling the effective capacity of each layer.

6 Training Configuration

6.1 Experiment 1

BCEWithLogitsLoss computes the binary cross-entropy directly from logits using the numerically stable log-sum-exp form. For a single prediction:

$$\ell(y, \hat{z}) = -\left[y \log \sigma(\hat{z}) + (1 - y) \log(1 - \sigma(\hat{z}))\right] \tag{10}$$

Table 4: Training hyperparameters — Experiment 1.

Parameter	**Value**
Optimiser	Adam
Learning rate	10^{-3}
Weight decay	None
Batch size	128
Epochs	50 (fixed)
Early stopping	None
Loss function	BCEWithLogitsLoss (no smoothing)
LR schedule	None (constant)
Multi-task	No

With a fixed 50-epoch schedule and no learning rate decay, the model reaches a point where the training loss continues to decrease while validation loss plateaus and rises — a clear sign of overfitting with no mechanism to stop it.

6.2 Experiment 2

Table 5: Training hyperparameters — Experiment 2.

Parameter	**Value**
Optimiser	AdamW
Learning rate	5×10^{-4}
Weight decay	10^{-5}
Batch size	256
Max epochs	100
Early stopping	Patience = 15 epochs
Loss function	Label-smoothed BCEWithLogitsLoss
Label smoothing (ε)	0.001
LR schedule	Cosine annealing ($\eta_{\min} = 10^{-6}$)
Auxiliary loss weight	0.3
Gradient clipping	1.0 (global norm)

6.2.1 AdamW vs. Adam

AdamW [4] decouples weight decay from the gradient update. Standard Adam with L2 regularisation scales the weight penalty by the adaptive learning rate, which reduces its effectiveness on parameters with large gradients. AdamW applies weight decay uniformly:

$$\theta_{t+1} = \theta_t - \alpha \cdot \hat{m}_t / (\sqrt{\hat{v}_t} + \epsilon) - \alpha\lambda\theta_t \tag{11}$$

This provides more reliable regularisation and better generalisation on tabular datasets.

6.2.2 Label Smoothing

With extreme class imbalance, the model is incentivised to push logits for negative-class predictions toward $-\infty$. This leads to overconfident predictions and poor calibration. Label smoothing replaces hard binary targets with soft targets:

$$\tilde{y} = y \cdot (1 - \varepsilon) + 0.5 \cdot \varepsilon \tag{12}$$

With $\varepsilon = 0.001$, a true positive becomes 0.9995 and a true negative becomes 0.0005. This keeps logits bounded, reduces overconfidence, and provides mild regularisation without significantly altering the effective training signal.

6.2.3 Cosine Annealing

The learning rate follows a cosine schedule over $T_{\max}$ epochs:

$$\eta_t = \eta_{\min} + \frac{1}{2}(\eta_{\max} - \eta_{\min}) \left(1 + \cos\left(\frac{\pi t}{T_{\max}}\right)\right) \tag{13}$$

This smoothly decays the learning rate, allowing large exploratory steps early in training and fine-grained refinement near convergence. Compared to a constant learning rate, cosine annealing consistently improves final validation performance.

6.2.4 Early Stopping

Training halts if the validation BCE loss does not improve for 15 consecutive epochs. The model weights from the best validation epoch are restored before evaluation. This prevents the late-epoch overfitting observed in Experiment 1, where the training loss continued to decrease while validation loss increased from epoch 10 onwards.

6.2.5 Total Loss

The combined training loss is:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{scored}} + \lambda_{\text{aux}} \cdot \mathcal{L}_{\text{aux}} \tag{14}$$

where $\mathcal{L}_{\text{scored}}$ is the label-smoothed BCE on the 206 competition targets, $\mathcal{L}_{\text{aux}}$ is the label-smoothed BCE on the 402 non-scored targets, and $\lambda_{\text{aux}} = 0.3$ balances the auxiliary contribution. The weight 0.3 was chosen so the auxiliary signal regularises without dominating — too high and the backbone specialises for the non-scored biology; too low and the multi-task benefit vanishes.

7 Results

7.1 Experiment 1

Table 6 summarises the per-fold results for Experiment 1.

The OOF evaluation across all folds gives:

Table 6: Experiment 1 — per-fold results. Val Loss is the minimum validation BCE loss achieved across 50 epochs.

Fold	Best Val BCE Loss	Best Val F1 (macro)
0	0.0206	0.0839
1	0.0191	0.0770
2	0.0227	0.0685
3	0.0186	0.0800
4	0.0227	0.0711
Mean	**0.0207**	**0.0761**

- OOF Macro F1: 0.0406
- OOF Flat Accuracy: 0.9964

The high flat accuracy is misleading: since $\sim 99.6\%$ of all label cells are zero, a model that predicts zero for everything achieves near-perfect accuracy. The low macro F1 (4%) reflects the model's failure to identify true positives in the severely imbalanced label space.

The training curves show a textbook overfitting pattern: training loss drops steadily from epoch 1 to 50, while validation loss bottoms out around epoch 5–10 and rises thereafter. By epoch 50, training F1 reaches ~ 0.60 while validation F1 remains below 0.08, indicating the model memorised training samples rather than generalising.

7.2 Experiment 2

Experiment 2 introduces a set of coordinated improvements. A 2-epoch smoke test verified the architecture and data pipeline are correct; the log loss dropped from 0.059 (random initialisation) to 0.026 after just two epochs, confirming the improved preprocessing and architecture produce faster, more stable learning than Experiment 1's equivalent early training phase.

Full 5-fold training is required to obtain final results; Table 7 will be updated upon completion.

8 Future Work and Takeaways

8.1 Immediate Next Steps

1. **Run Experiment 2 to completion.** Obtain the 5-fold OOF log loss and compare against the trivial baseline (0.022) and Experiment 1's estimate (~ 0.021).
2. **Hyperparameter search.** The current Experiment 2 hyperparameters (hidden dims, dropout, learning rate, label smoothing, aux weight) were set by informed prior rather than search. A systematic grid or Bayesian search over these could yield meaningful gains.
3. **Threshold tuning.** The label smoothing value $\varepsilon = 0.001$ and aux weight $\lambda_{\text{aux}} = 0.3$ are reasonable defaults. Tuning them on OOF predictions (e.g. by minimising OOF log loss) should be explored.

Table 7: Model comparison summary.

Aspect	Experiment 1	Experiment 2
Input dim	873	1,560
Hidden dims	512, 256	1024, 512, 256
Batch normalisation	No	Yes
Activation	ReLU	SiLU
Residual block	No	Yes
Dropout rate	0.5	0.3
Multi-task targets	206	206 + 402
Optimiser	Adam	AdamW
LR schedule	None	Cosine annealing
Label smoothing	No	$\varepsilon = 0.001$
Early stopping	No	Patience = 15
Feature normalisation	None	QuantileTransformer
Dim. reduction	None	PCA (95% variance)
Best Val BCE Loss	0.0207	TBD
OOF Macro F1	0.0406	TBD
OOF Log Loss	$\sim$0.021*	TBD

*Estimated; competition metric was not tracked during Experiment 1 training.

8.2 Modelling Improvements

1. **Ensemble with tree-based models.** The top Kaggle solutions combined MLP predictions with LightGBM and/or XGBoost. Tree models capture feature interactions differently from neural networks, and blending the two typically yields a significant improvement over either alone.

2. **TabNet.** TabNet [5] is an attention-based tabular model that performs instance-wise feature selection. It has shown strong results on this exact dataset in competition settings and would be a natural third model for an ensemble.

3. **Deeper or wider residual architectures.** Adding more residual blocks, or increasing hidden dimensions, may capture higher-order gene–MoA interactions. This must be balanced against overfitting risk given the dataset size.

4. **Variance-based feature selection.** Some gene features may have near-zero variance across samples and contribute only noise. A variance threshold filter before PCA could improve the quality of the principal components.

5. **Prediction calibration.** Since the metric is log loss, calibration of the output probabilities matters. Platt scaling or temperature scaling applied to the OOF predictions may improve the final score without retraining the model.

6. **Rank-based ensembling.** Rather than averaging probabilities across folds, averaging the rank-normalised predictions before converting back to probabilities can reduce the influence of any single fold's calibration errors.

8.3 Key Takeaways

1. **Track the right metric.** Experiment 1 optimised and reported BCE loss and F1. Neither is the competition metric. F1 in particular is misleading on extreme class imbalance — a 99.6% flat-zero accuracy can coexist with a near-zero F1. Always monitor mean column-wise log loss on OOF predictions during development.

2. **Preprocessing matters as much as architecture.** The move from raw features to quantile-normalised + PCA features increased input dimensionality from 873 to 1,560 while providing a cleaner signal. Two epochs of Experiment 2 already outperforms the best checkpoint of Experiment 1 in log loss terms.

3. **Batch normalisation is essential at depth.** Without it, the model in Experiment 1 showed aggressive overfitting from epoch 10 onwards. Batch normalisation stabilises gradient flow and allows effective use of greater depth and width.

4. **Drug-level cross-validation is non-negotiable.** Sample-level splits inflate validation scores by placing the same drug in both train and validation. Any metric reported without drug-level grouping cannot be trusted as a generalisation estimate.

5. **Free signal from non-scored targets.** The 402 non-scored labels describe the same biological response as the 206 scored ones. Ignoring them (as in Experiment 1) leaves a substantial amount of training signal unused. Multi-task learning with auxiliary targets is a low-cost, high-benefit technique for this dataset.

9 Conclusion

We presented two experiments on the LISH-MoA drug classification problem. Experiment 1 established a working baseline MLP that confirmed the pipeline is correct end-to-end, but suffered from overfitting, suboptimal feature representation, and a mismatch between the monitored metric (BCE, F1) and the actual competition objective (mean column-wise log loss). Experiment 2 systematically addressed all identified weaknesses: richer preprocessing, a deeper and better-regularised architecture, multi-task learning, and a training loop aligned with the competition metric. Early validation results confirm faster convergence and better calibration. The Experiment 2 framework is the recommended foundation for all subsequent model development.

References

[1] Ioffe, S. and Szegedy, C. (2015). *Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift.* ICML 2015.

[2] He, K., Zhang, X., Ren, S., and Sun, J. (2016). *Deep Residual Learning for Image Recognition.* CVPR 2016.

[3] He, K., Zhang, X., Ren, S., and Sun, J. (2015). *Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification.* ICCV 2015.

[4] Loshchilov, I. and Hutter, F. (2019). *Decoupled Weight Decay Regularization.* ICLR 2019.

[5] Arik, S. Ö. and Pfister, T. (2021). *TabNet: Attentive Interpretable Tabular Learning.* AAAI 2021.